PROJECT REPORT: MINLINK ADVANCED URL SHORTENER

Subject: Web Application Development using Flask, SQLAlchemy, and Bootstrap

1. Project Overview

The **MinLink URL Shortener** is a web-based utility designed to transform long, cumbersome URLs into manageable, short links. Beyond simple redirection, the application provides an "Advanced User" experience by integrating a secure authentication system and a persistent database. This allows users to maintain a personal history of their shortened links, ensuring that important data is never lost after a session ends.

1.1 Objectives

- **Core Functionality:** Convert any valid URL into a unique 6-character short code.
- **User Persistence:** Allow users to create accounts and log in to save their link history.
- **Interface Efficiency:** Provide a minimalist, high-speed interface with one-click copy functionality.
- **Security:** Enforce strict validation rules for user identification and password storage.

2. Technical Architecture

The application follows a modular architecture, separating data management from the user interface.

2.1 The Tech Stack

1. **Backend:** Python 3.10+ with the **Flask** micro-framework.
2. **Database Engine: SQLite**, chosen for its zero-configuration, file-based storage.
3. **ORM: SQLAlchemy**, providing a Pythonic way to interact with the database without writing raw SQL.
4. **Frontend: Bootstrap 5**, providing a responsive CSS framework for a professional, minimalist look.
5. **Authentication: Flask-Login**, used for managing user sessions and protecting private routes.

3. Database Design (Schema)

The application utilizes a relational database to link data points together.

3.1 User Table

Stores credentials for registered members.

- `id`: Primary Key (Integer).
- `username`: Unique String (Length restricted to 5-9 characters).
- `password`: Hashed String (Stored using PBKDF2 for security).

3.2 Link Table

Stores the mapping between original and shortened URLs.

- `id`: Primary Key.
- `original_url`: The destination link provided by the user.
- `short_code`: The unique 6-character identifier.
- `user_id`: Foreign Key linking the URL to the specific User who created it.

4. System Implementation & Workflow

4.1 Registration Logic (The 5-9 Constraint)

To prevent common bot registrations and ensure a specific user experience, we implemented a custom validation layer in the /signup route. If the input length is L, the system enforces 5 < L <9. If L falls outside this range, the system triggers a Flask Flash message, preventing the database commit and notifying the user immediately.

4.2 Short Link Generation

The "shortening" is achieved via a randomized algorithm using the string and random Python modules. It selects characters from the set {A-Z, a-z, 0-9}.

The probability of a collision (two users getting the same code) is extremely low, as there are 62^6 (over 56 billion) possible combinations.

4.3 Redirection Engine

The redirection is handled by a dynamic route decorator: `@app.route('/<short_code>')`. When a short link is accessed, the application performs a database lookup. If the code exists, the backend sends an HTTP 302 status code, redirecting the user's browser to the stored `original_url`.

5. UI/UX Design Philosophy

The frontend was built using **Template Inheritance**.

- **Base Layout (`base.html`):** Contains the core HTML structure, Bootstrap CDN links, and the navigation bar. This ensures that every page feels like part of the same application.
- **User Dashboard:** Designed as a "Control Center." The top section features a prominent URL input field, while the bottom section displays a tabular "History" view.

- **Interactive Elements:** I used the **JavaScript Clipboard API** to ensure that "Advanced Users" can copy links instantly without manual selection, reducing friction in the workflow.

6. Security Measures

- **Password Hashing:** Using `werkzeug.security`, passwords are salt-hashed. This means even the database administrator cannot see the actual passwords.
- **Protected Routes:** Using the `@login_required` decorator, we ensure that the Dashboard and Shortening logic cannot be accessed by unauthenticated guests.
- **Automatic Table Creation:** The script includes a `with app.app_context(): db.create_all()` block, which automatically detects if the database is missing and builds it on startup, preventing runtime crashes.

7. Conclusion

The MinLink application successfully integrates a robust backend with a clean frontend to solve the problem of long, unmanageable URLs. By focusing on a minimalist design and strict data validation, the project provides a professional-grade tool suitable for advanced users who require link persistence and account security.